Observability-Aware Trust-Based Grayhole Detection for Underwater Acoustic Networks

Appaji Nagaraja Dheeraj
Dept. of Computer Science and Engineering
National Institute of Technology Karnataka
Surathkal, Mangalore, India
appajidheeraj.241cs110@nitk.edu.in

Aryan Bokolia
Dept. of Computer Science and Engineering
National Institute of Technology Karnataka
Surathkal, Mangalore, India
aryanbokolia.241cs111@nitk.edu.in

Karthikeya Gupta
Dept. of Computer Science and Engineering
National Institute of Technology Karnataka
Surathkal, Mangalore, India
karthikeyagupta.241cs117@nitk.edu.in

***Abstract*—Underwater networks provide new possibilities that can greatly improve our ability to observe and protect the oceans. Underwater acoustic sensor networks are frequently deployed in remote and unattended waters, making them vulnerable to various security threats. Applications based on these networks, such as pollution monitoring, offshore infrastructure inspection and coastal surveillance to name a few, require reliable delivery of every measurement. Various routing protocols have been developed to carry this data over several hops to a surface sink. However, these protocols often place a low priority on security, and a compromised relay that silently drops part of the traffic, known as a grayhole, can degrade the entire network. The classic watchdog defence cannot tell such a relay apart from an honest one whose forwards were lost to collisions, half-duplex deafness or long acoustic links, and it ends up accusing honest nodes. This paper takes a look at the grayhole attack, examines a model of the attack as well as an observability-aware, trust-based detection mechanism created in UnetStack, and compares it with a fixed-threshold watchdog and a channel-aware detector.**

***Index Terms*—Underwater acoustic sensor network(UASN), Grayhole attack, Selective Forwarding(SF), Trust, Watchdog, UnetStack**

I. Introduction

A. Basics of Underwater Acoustic Sensor Networks

Underwater acoustic sensor networks are now emerging as a hot topic among researchers due to a surge of novel applications in ocean sampling, environmental monitoring, offshore exploration, disaster prevention and tactical surveillance. An Underwater Acoustic Sensor Network (UASN) consists of sensors placed under water that sense the environment and communicate using sound, since radio waves are absorbed within a few metres of sea water. Each device that can send, receive and relay data is called a *Node*. One special node, the *sink*, is usually a surface buoy that collects the data and passes it on to the shore.

Sound travels at about 1500 m/s in water, nearly five orders of magnitude slower than radio. The acoustic channel faces a number of challenges, some of which are given in [1]: long and variable propagation delay, low and distance-dependent bandwidth, high bit-error rates, noise from ships, boats and marine life, and nodes that run on batteries that are hard to replace. Moreover, most acoustic modems are half-duplex: a node cannot hear while it is transmitting. A sensor that is far from the sink therefore depends on other nodes to relay its packets hop by hop.

Attackers can use these issues to their advantage. Since packets are lost often even in an honest network, a compromised relay can drop packets without causing much suspicion. Nodes are deployed unattended, so physically capturing one and loading malicious code into it is feasible. Unlike a terrestrial network, there is no cheap way to inspect a node that sits hundreds of metres below the surface.

As a result, security is a major issue faced by UASNs currently. Light-weight detection schemes that do not mistake the channel for an attacker are the need of the hour. This paper looks into detecting the grayhole attack, a form of selective forwarding, at the network layer.

B. Selective Forwarding and the Grayhole Attack

In a multi-hop network, the role of a routing node is to receive packets from its neighbours and forward them towards the sink. A selective forwarding attack [2] occurs when such a node forwards only some of the packets and drops the rest. When every packet is dropped, the node is called a *blackhole*; it is easy to find, because traffic through it stops completely. A *grayhole* drops only a fraction of the packets, for example 30%, so its loss blends with the normal loss of the acoustic channel. The attack raises the packet loss rate and lowers the throughput and reliability of every route that passes through the malicious node. Since the busiest relays carry the traffic of many sensors, a single grayhole can affect a large part of the network.

There are 2 ways by which a malicious node can attack the network, as explained in [2], [3].

1) Insider Attack
2) Outsider Attack

An *insider attack* is carried out by a node that belongs to the network and holds its keys, for example a captured sensor running malicious code. Since it takes part in routing legitimately, cryptography alone cannot stop it from dropping packets. An *outsider attack* is carried out by a node that does not belong to the network, for example by jamming or by replaying packets. Outsider attacks can be limited with authentication. The grayhole considered in this paper is an insider.

C. Types of Selective Forwarding

Selective forwarding appears in several forms [2], [4]:

- **Blackhole:** the node drops every packet that it should forward.
- **Grayhole:** the node drops packets with a given probability, or only those of chosen sources, so the loss looks like that of a poor channel.
- **On–off attack:** the node behaves well for a while to build trust and then drops packets, repeating the cycle.
- **Denial of Service:** the node drops control packets, such as route updates, so that its neighbours lose their routes.

There are a variety of solutions available for these attacks, but often these require acknowledgements from every hop, extra probe packets or cryptographic exchanges, which use high amounts of energy and bandwidth. This is a luxury that cannot be afforded in the case of UASNs. Hence, the approach in this paper only listens to traffic that is already sent, and it was chosen taking this and the feasibility of implementation into account.

The rest of the paper is organised as follows: Section II presents a review on various related works of literature. Section III presents the design of the grayhole attack model and the detection approach. Section IV explains the implementation of both in UnetStack. Section V presents results of a performance evaluation done on both implementations, together with a comparison against the baselines. Section VI concludes the paper and provides suggestions for future work.

II. Related Work

The authors of [1] elaborate on the architecture of UASNs and the research challenges of every layer, from the slow and lossy acoustic channel up to routing. The paper authored by [2] talks about attacks on routing in sensor networks, and is one of the first to define selective forwarding and to show why it is hard to counter. Domingo in [3] lists the attacks that apply to underwater networks, including selective forwarding, sinkhole and wormhole attacks, and the countermeasures available for each. These papers however only presented general-purpose solutions, and they do not describe how a node should decide that a neighbour is malicious.

The paper by [4] contains an extensive literature survey of security in underwater acoustic networks, covering every layer from the physical layer to the application. It points out that most terrestrial defences assume a reliable channel and cheap communication, neither of which holds under water.

Marti *et al.* in [5] introduced the *watchdog*: a node that has sent a packet to its next hop keeps listening, and if it does not overhear the next hop forwarding the packet within a timeout, it counts a miss. A node whose misses exceed a threshold is reported as misbehaving. The method is simple, distributed and needs no extra packets. Some possible drawbacks noticed were, as the authors themselves point out, ambiguous collisions, receiver collisions and partial dropping: a miss can be caused by a collision at the monitor or at the relay, not only by the relay.

Shila *et al.* in [6] improve on this for grayhole attacks in wireless mesh networks with a Channel-Aware Detection (CAD) scheme. CAD estimates the normal loss that the channel and medium-access collisions would cause, and accuses a node only when its observed loss is clearly higher than this estimate. It combines upstream and downstream monitoring with probe and acknowledgement packets. Hence, there is a trade-off involved here: better accuracy is paid for with extra control traffic, which is costly under water.

The group who authored [7] did another study on trust in UASNs and built a trust model from several metrics, namely link, data and energy trust. Liu *et al.* [8] use an adaptive trust evaluation to detect abnormal nodes, and Han *et al.* [9] combine trust with fuzzy logic to choose reliable relays in layered underwater routing. These trust models show that trust is useful under water, but they compute trust from counts of successful interactions and do not ask whether a failed interaction could have been observed in the first place.

Chandavarkar and Suresh in [10] demonstrated the selective forwarding attack and a detection mechanism in UnetStack3. Their work showed that the attack can be built and studied on an underwater network stack. Of the methods above, the fixed-threshold watchdog of [5] is the simplest to implement and is the style of detector used for this demonstration. It is distributed and needs no extra packets. Hence, it was chosen as the baseline of this paper. A CAD-style detector adapted from [6] is used as a second comparator. Both comparators are adaptations: our watchdog does not reproduce the destination-timeout details of [10], and our CAD-style detector keeps only the downstream channel and collision estimate of [6]. Removing a detected node can consume a lot of energy and can cut off the nodes behind it. Hence, our approach only detects grayholes and routes traffic around them where another path exists.

Most of the work above was tested on general network simulators designed for terrestrial or radio networks. UnetStack [11] is an agent-based network stack and simulator for underwater networks, in which the same agents can run on real modems. There is a dearth of papers covering simulations in UnetStack, especially in the domain of security. This paper seeks to address that by providing one such implementation, built with the physical-layer notifications that UnetStack provides [12].

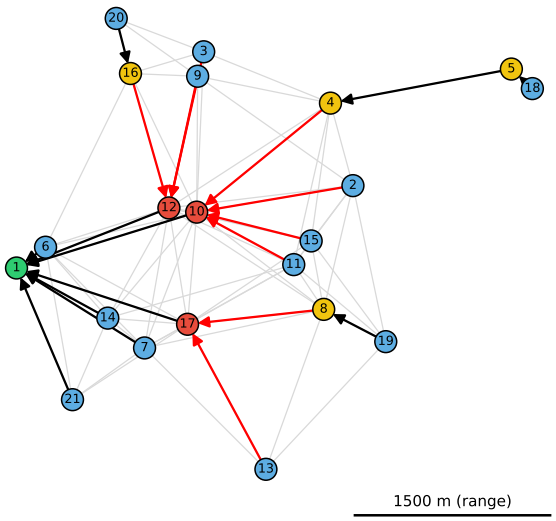


Fig. 1. Network topology to simulate the grayhole attack (test network 201).

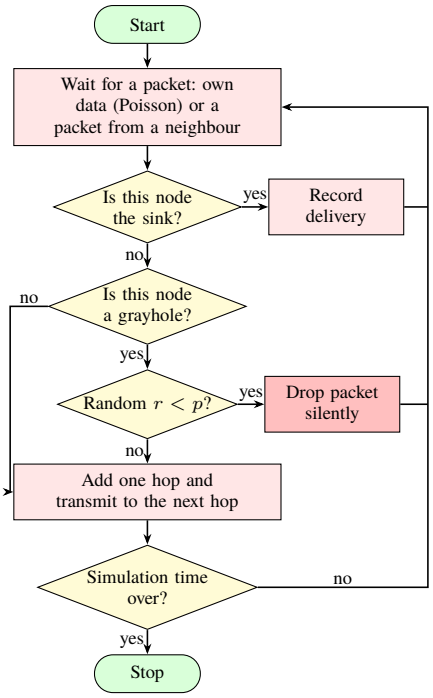


Fig. 2. Sequence of the attack model.

III. Design of the Grayhole Attack and its Detection

A. Attack Model

A 20-node network created on UnetStack is used to demonstrate the attack. Twenty sensors are placed at random positions in a 4 km $\times$ 4 km field on one side of the sink, and the layout is redrawn until every sensor can reach the sink. Every sensor generates data for the sink, and each one sends its packets to the nearest neighbour that is one hop closer to the sink. The three relays that carry the most traffic are made malicious. One of the ten test networks is given in Figure 1.

In the figure, node 1 is the sink, coloured green. The sensors that only send their own data are coloured blue, the honest relays are coloured yellow and the three grayholes, nodes 10, 12 and 17, are coloured red. The arrows show the route that each node uses towards the sink. The arrows in black are routes into honest nodes, and the arrows in red are routes into a grayhole, along which part of the traffic is dropped. The thin grey lines join nodes that are within the 1500 m communication range of each other; along these lines a node can overhear its neighbour. The scale bar shows the communication range. Nodes 7 and 14 are only about 100 m apart and are drawn slightly apart so that both can be read.

Initially, every node will start sending data packets to its next hop at random times. When a grayhole receives a packet that it should forward, it will draw a random number between 0 and 1. If the number is below the drop probability p (0.3 in most of the experiments), the packet will be dropped silently; otherwise it will be forwarded as usual. The grayhole still sends its own data and still answers every request, so it looks like an honest node to its neighbours. A flowchart showing the course of the program is given in Figure 2.

```
Repeat
  send packet P to next hop B; sent[B]++
  start timer for P
  if (forward of P by B overheard before timeout)
    // B behaved well for P
  else
    miss[B]++                  // any silence counts
  if (sent[B] >= MIN_OBS && miss[B]/sent[B] > 0.3)
    accuse(B)                  // fixed threshold
Until No transmission
```

Fig. 3. Baseline fixed-threshold watchdog algorithm (after [5]).

B. Detection Approach

The detection approach starts from the watchdog of [5], which is the baseline of this paper. It is distributed: every node that sends a packet watches its own next hop. The following rules are followed by each monitor node:

Rule 1: *After sending a packet to the next hop, listen for its forward; a forward that is not heard in time is a miss.*

Rule 2: *If the fraction of misses exceeds a fixed threshold (30%), accuse the next hop.*

Given in Figure 3 is the algorithm of the baseline watchdog.

In the baseline, every silence is blamed on the next hop. Under water, however, a forward can be missed although the relay is honest: the monitor itself was transmitting and was deaf (half-duplex), the packet collided at the relay so the relay never received it, or the long acoustic link lost the packet or its forward. These losses grow with traffic, so with a fixed threshold the baseline either accuses honest relays or misses grayholes, and routing around wrongly accused relays forces detours that lower delivery.

Taking into account what an acoustic node in UnetStack can actually observe, the following modifications were made:

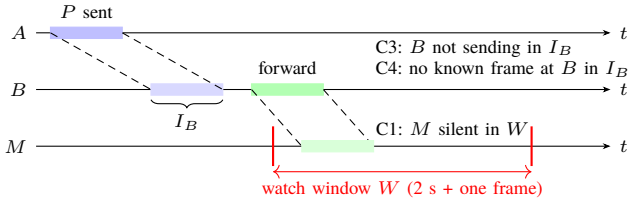


Fig. 4. Watch window of monitor M for relay B. Dashed lines show propagation at 1500 m/s.

- **Observability:** every neighbour of a relay watches it, and a watch window counts only if the monitor could have heard the forward. Windows hidden by the monitor's own transmission or by a known collision at the relay are ignored, instead of being counted as misses.
- **Trust from the expected loss of each link:** for each link the monitor computes, from the node distances and the normal loss of the link's distance area (near, medium or far), how many forwards an honest relay should show. Trust is the number of forwards heard divided by the number expected, combined over all monitors.
- **Dynamic threshold and confirmation:** a relay is flagged only when its shortfall is significant for the current network, and it becomes a suspect only after two flagged rounds in a row. Confirmed grayholes leave the normal-loss baseline, so further grayholes stand out.

The timing of one watch is shown in Figure 4. Let M be the monitor, A the node that sent packet P and B the relay. From the known positions and the speed of sound c, M works out when P was arriving at B (interval I_B) and when B's forward should reach M (window W). W is as long as the forward allowance of 2 s plus one frame.

A window is kept only if three conditions hold. **C1:** M did not transmit during W. **C3:** M did not hear B transmitting while P was arriving at B, since B was then deaf. **C4:** no frame known to M, whether its own or one it decoded from another neighbour, reached B during I_B, so P did not collide at B. The same conditions are applied to heard and missed forwards. A fourth candidate, a corrupted frame at M (C2), is deliberately not used, because the relay's own forward can cause it while a drop never does: excluding such windows would remove honest forwards more often than drops.

For each kept window, the chance that an honest forward reaches M through the channel alone is $g = (1-e(d_{AB}))(1-e(d_{BM}))$, where $e(d) = 0.15\,(d/1500)^2$ is the frame loss at distance d. The loss that the channel does not explain, mainly collisions, is learned from the network itself: for each distance area it is the median loss of the *other* relays' links, so the relay being judged cannot hide in its own baseline. The expected forwards on a link are then $E = G(1-\kappa_{\text{area}})$, where G is the sum of g over its windows. The trust of relay B is the total heard divided by the total expected over all its monitors, $T_B = \sum O / \sum E$, and its estimated drop rate is $1 - T_B$. This is the same as a weighted average of every monitor's trust, in which each monitor counts by the number of forwards it expected. Given in Figure 5 is a flowchart depicting the sequence of the detection strategy.

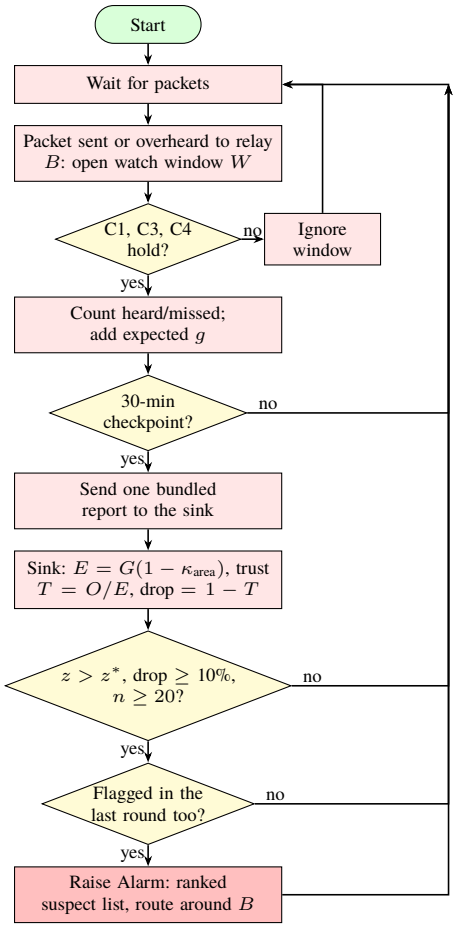


Fig. 5. Sequence of the detection strategy.

Every 30 minutes the sink runs the detector on the latest report of every link. A relay is flagged when three conditions hold: it was judged on at least 20 windows, its estimated drop rate is at least 10%, and its deviation z, the shortfall of heard forwards measured in standard deviations, is above a threshold z^*. The threshold is dynamic: it grows with the number of judged relays, and z is first rescaled by how widely honest relays spread in the current round. When several grayholes sit in one area, they raise the area's normal loss and hide one another; the detector therefore removes relays with at least 10% estimated drop, and confirmed suspects, from the area medians and repeats the estimate. A relay that is flagged in two rounds in a row becomes a suspect, and the sink routes traffic around it wherever another path exists.

IV. Implementation of the Grayhole Attack and Detection in UnetStack

The implementation uses UnetStack 7.0.0 (community edition) with Java 8. Each node runs one agent, `MonitorNode`, which generates data, relays packets, acts as a grayhole when told to, and monitors its neighbours. The acoustic channel is UnetStack's `ProtocolChannelModel` extended in

DistanceLossChannel so that the chance of decoding a frame falls with distance as $1 - 0.15\,(d/1500)^2$; far links therefore lose more frames than near ones even when every node is honest. The class diagram of the implementation is given in Figure 6.

A. Attack Model

The agent used has three main functions: **startup()**, **generate()** and **onPacket()**, along with a utility function transmit(pkt), which sends a packet to the current next hop using a TxFrameReq to the physical layer. A data packet is 8 bytes long: origin, sequence number, generation time and hop count.

1) startup(): The role of startup() is akin to that of the main() function in any regular computer program [11]. It is called once when the agent starts, finds the physical-layer agent, subscribes to its notifications, and adds the behaviours that run for the rest of the simulation.

```
startup() {
  phy = agentForService(PHYSICAL)
  subscribe(topic(phy))         // overhear every frame
  subscribe(topic(DATAGRAM))    // frames for this node
  if (genRate > 0)
    add PoissonBehavior(1/genRate) { generate() }
  add TickerBehavior(200 ms) { closeWindows() }
  add TickerBehavior(30 min) {
    snapshot(); sendReports(); onRound() }
}
```

2) generate(): It builds a new data packet with the node's address and the next sequence number and passes it to transmit().

3) onPacket(): This function carries out the attack. It is called for every packet addressed to the node. The sink records the delivery, or stores the report. Any other node forwards the packet, unless it is a grayhole and the random draw tells it to drop the packet.

```
onPacket(pkt) {
  if (addr == SINK) {
    if (isReport(pkt)) acceptReport(pkt)
    else delivered.add(pkt)
    return
  }
  if (dropProb > 0 && random() < dropProb)
    return                      // grayhole drops it
  pkt.hops++
  transmit(pkt)                 // forward to next hop
}
```

B. Detection Approach

The agent designed here has 3 primary functions for the monitor: **openWindow()**, **closeWindows()** and **relayClear()**, along with **sendReports()**. The detector itself is in the class Detectors, whose main function is **ours()**. Report transmissions are jittered over ten minutes to prevent packet collisions between reports.

1) openWindow(): When the node's own frame leaves (TxFrameNtf) or when it overhears a frame addressed to another node (RxFrameNtf), it opens a window for the relay. If a frame from relay B carrying the same packet identity is overheard while the window is open, the window is marked as heard.

```
openWindow(B, id, arrEnd, src, sender) {
  if (B == SINK || dist(addr, B) > RANGE) return
  back = dist(B, addr) / SOUND
  windows[B:id] = [heard: false, src, sender,
    I: [arrEnd - FRAME, arrEnd],
    W: [arrEnd + back, arrEnd + FWD_WAIT + FRAME + back]]
}
```

2) closeWindows(): Every 200 ms it closes the windows that have ended, checks the conditions and adds the outcome to one of 32 counters, so that every method can later be judged on the same evidence. The counter index keeps whether the monitor was the sender, which conditions failed and whether the forward was heard.

```
for (w in windows where now > w.W.end + 50 ms) {
  c13 = !overlaps(myTx, w.W) && !overlaps(heardTx[B], w.I)
  c2  = !overlaps(badFrames, w.W)   // kept, not used
  c4  = relayClear(w)
  i = idx(w.sender, c13, c2, c4, w.heard)
  ev[B][i]++
  expect[B][i] += (1 - e(d(src,B))) * (1 - e(d(B,addr)))
  remove(w)
}
```

3) relayClear(): It checks C4. A frame sent by node C during $[t_0, t_1]$ is at B during $[t_0, t_1]$ shifted by $d(C, B)/c$. If such a known frame overlaps I_B, the packet may have collided at B and the window is not observable.

```
relayClear(w) {
  for (C, frames in heardTx + [addr: myTx]) {
    if (C == w.src || C == B || dist(C, B) > RANGE)
     continue
    shift = dist(C, B) / SOUND
    if (any frame f: f + shift overlaps w.I)
      return false              // possible collision at B
  }
  return true
}
```

4) sendReports(): At each checkpoint the monitor freezes its counters and bundles them into one report frame: a 12-byte header and one 17-byte entry per watched relay (relay, windows, misses and expected forwards), at most six entries in the 128-byte frame. The report travels to the sink over the same lossy network as the data, and can be lost or dropped by a grayhole. The sink keeps the latest delivered report of every link; since the counters are cumulative, a lost report only delays evidence.

5) ours(): This is the detector at the sink. The following are the parameters used by ours():

- **MIN_ROW** – a link joins the estimate with at least 5 windows.
- **MIN_OBS** – a relay is judged on at least 20 windows.
- **S_MIN** – minimum estimated drop rate to flag, 10%.
- **ALPHA** – false-alarm setting, 10^{-4}, chosen on separate tuning networks.
- **CHI2_MEDIAN** – 0.455, the median of z^2 for an honest relay, used to measure the spread of a round.

Given below is the gist of the UnetStack code for the mechanism.

```
for (link i with n_i >= MIN_ROW)
  kappa_i = max(0, 1 - O_i/G_i)  // unexplained loss
zone = near / medium / far tertile of link distance
for (relay B) {
  kArea = median(kappa of OTHER relays' links in zone)
  E_i = G_i * (1 - kArea)    // expected forwards
```

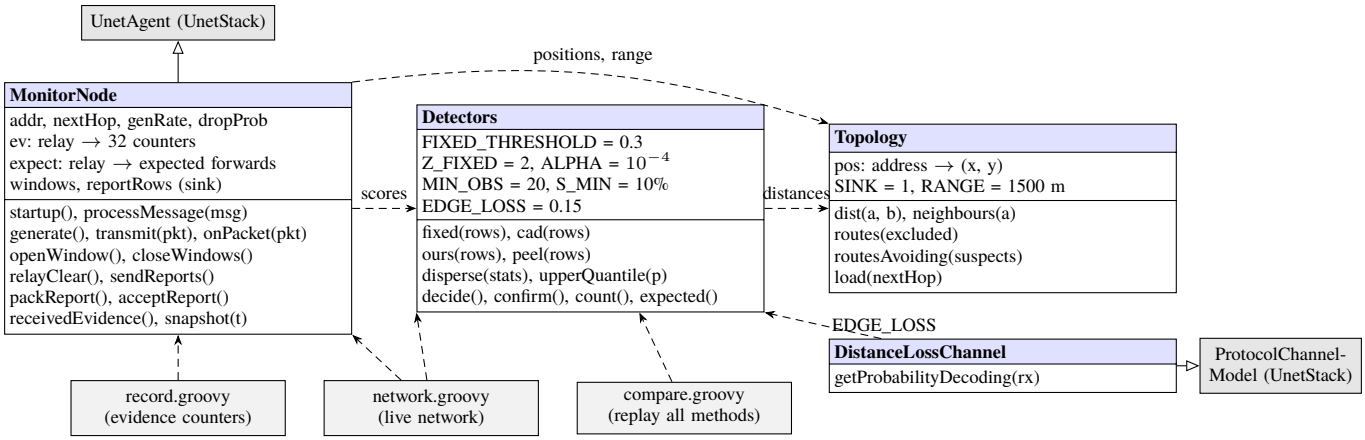


Fig. 6. Class diagram of the implementation. Solid arrows with open heads show inheritance, dashed arrows show use.

```
  trust = sum(O_i) / sum(E_i); drop = 1 - trust
  z = (sum E - sum O) / sqrt(sum E_i (1 - E_i/n_i))
}
z = z / sqrt(phi)             // spread of this round
zStar = upperQuantile(ALPHA / judgedRelays)
flag B if n >= MIN_OBS && drop >= S_MIN && z > zStar
```

The function **peel()** repeats ours() at most three times, leaving out relays with at least 10% estimated drop and confirmed suspects from the area medians, until the set is stable. This code repeats every 30 minutes till there is no more transmission.

6) onRound(): At the sink, after every round, the flagged set is confirmed and the routes are updated. A suspect stays one until the administrator reviews it.

```
onRound(t) {
  rows = receivedEvidence()   // delivered only
  flagged = peel(rows, ALPHA, suspects).flagged
  suspects += flagged.intersect(lastFlagged)
  lastFlagged = flagged
  if (suspects changed)
    nextHop = routesAvoiding(suspects)
}
```

The function `routesAvoiding()` gives every node a shortest route that avoids the suspects; a node with no such route keeps its old route, since a lossy path is better than none. The two baselines use exactly the same agent, reports, confirmation and response, so only the detection rule differs.

V. Results

All experiments run in one pipeline (`run.sh`): `record.groovy` simulates the networks and stores every monitor's counters, `compare.groovy` replays the sink's decisions for every method on exactly the same counters, `network.groovy` runs the live network in which reports, detection and rerouting all happen inside the simulation, and a Python script draws the graphs. The simulation parameters are as follows:

- **Network** – 20 sensors and one sink in 4 km × 4 km, range 1500 m, sound speed 1500 m/s; a 40-node and a sparse variant.
- **Traffic** – 0.001 to 0.01 packets/s per sensor; 0.005 unless stated.
- **Grayholes** – the 1, 2 or 3 busiest relays, dropping 30% of relayed packets (10–100% in a sweep).
- **Networks** – thresholds tuned on topologies 101–106 only; results on test topologies 201–210.
- **Duration** – 6 h recorded runs, 12 h live-network runs, one decision every 30 min.
- **Baselines** – fixed watchdog (miss rate $> 30\%$) and CAD-style ($z > 2$).

A. Attack Model

The first subsection examines the performance of the attack model. The strength of the attack is controlled by the number of grayholes, each dropping 30% of the packets it should forward. Figure 12 shows a graph showing the delivery ratio lost to the attack for different numbers of grayholes, measured against the same network without attack and without any defence. Each value is the average of 10 test networks. Each run lasted 12 hours, in which every sensor sent about 216 packets.

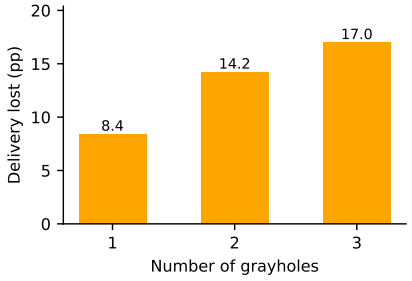


Fig. 12. Delivery lost for different numbers of grayholes (pp: percentage points of honest packets delivered).

It can be seen that the higher the number of grayholes, the more of the honest traffic is lost: 8.4, 14.2 and 17.0 percentage points of delivery for one, two and three grayholes. Furthermore, the loss grows more slowly with each added grayhole, since the busiest relay is attacked first and the later ones carry less traffic.

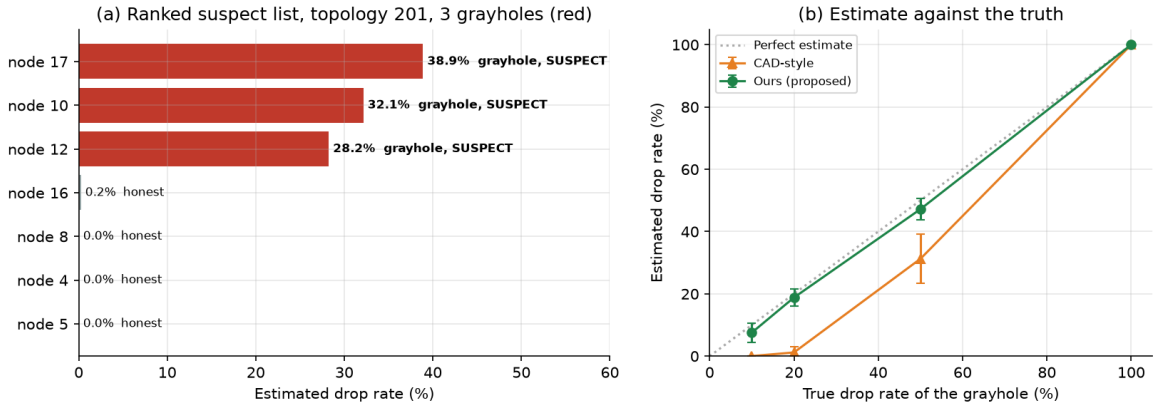


Fig. 7. (a) Ranked suspect list for test network 201 with three grayholes; (b) estimated against true drop rate.

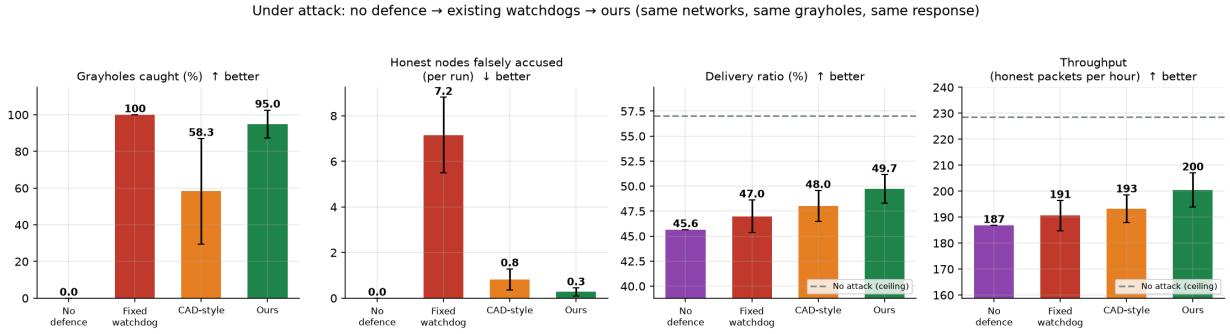


Fig. 8. Baseline vs proposed in the live network: grayholes caught, honest nodes accused, delivery ratio and throughput (12 h, ten test networks). The dashed line is the same networks without attack.

B. Detection Approach

Coming to the detection approach, the first question is whether the grayholes are found at all. Figure 13 shows the fraction of grayholes confirmed as suspects by the end of a 6-hour run, and Figure 14 the time at which every grayhole of a run had been confirmed, averaged over the runs in which this happened. Due to the random nature of packet drops and collisions, the time differs from run to run; since a relay needs two flagged rounds in a row, the earliest possible confirmation is after 60 minutes.

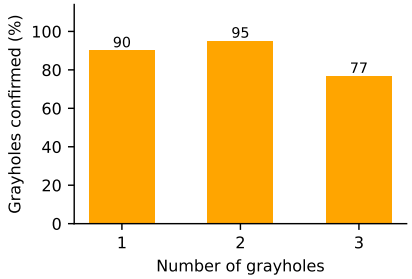


Fig. 13. Grayholes confirmed for different numbers of grayholes.

The detector confirmed 90% of single grayholes, 95% of grayholes in networks with two and 77% in networks with three. The detection time roughly rises from 97 to 140 minutes as grayholes are added, since several grayholes raise the normal loss of their area and hide one another until the baseline has been cleaned.

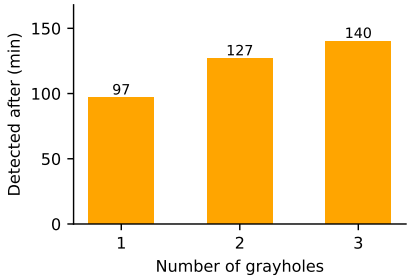


Fig. 14. Detection time for different numbers of grayholes.

Figure 7(a) is the list the administrator receives for the network of Figure 1. The three grayholes rank first, with estimated drop rates of 38.9%, 32.1% and 28.2%, and are confirmed; every honest relay is estimated at 0–0.2%. Figure 7(b) shows that the estimate follows the true drop rate: 7.6, 18.8 and 47.2% for true rates of 10, 20 and 50%.

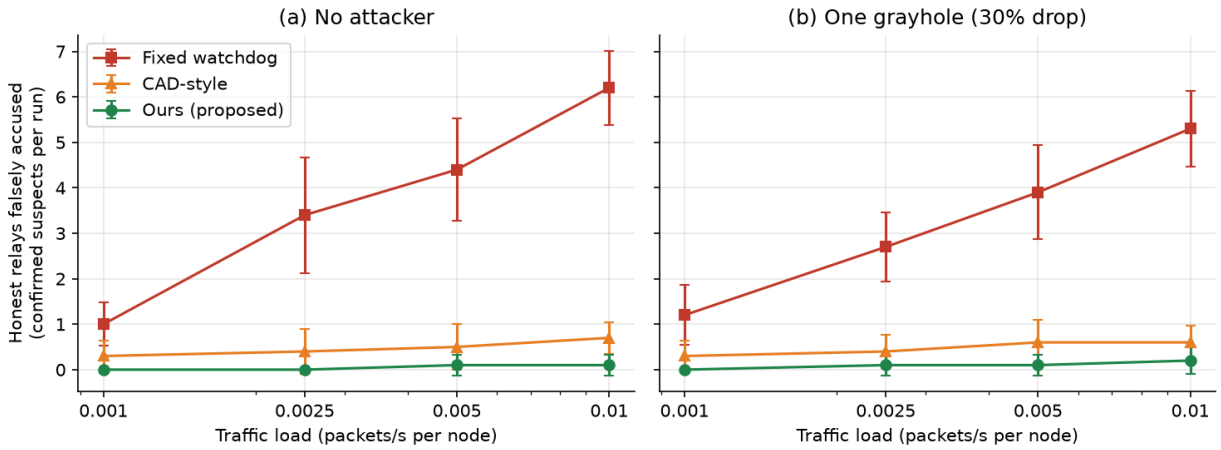


Fig. 9. Honest relays falsely accused per run for different traffic loads, each method with its own rule (6 h, ten test networks).

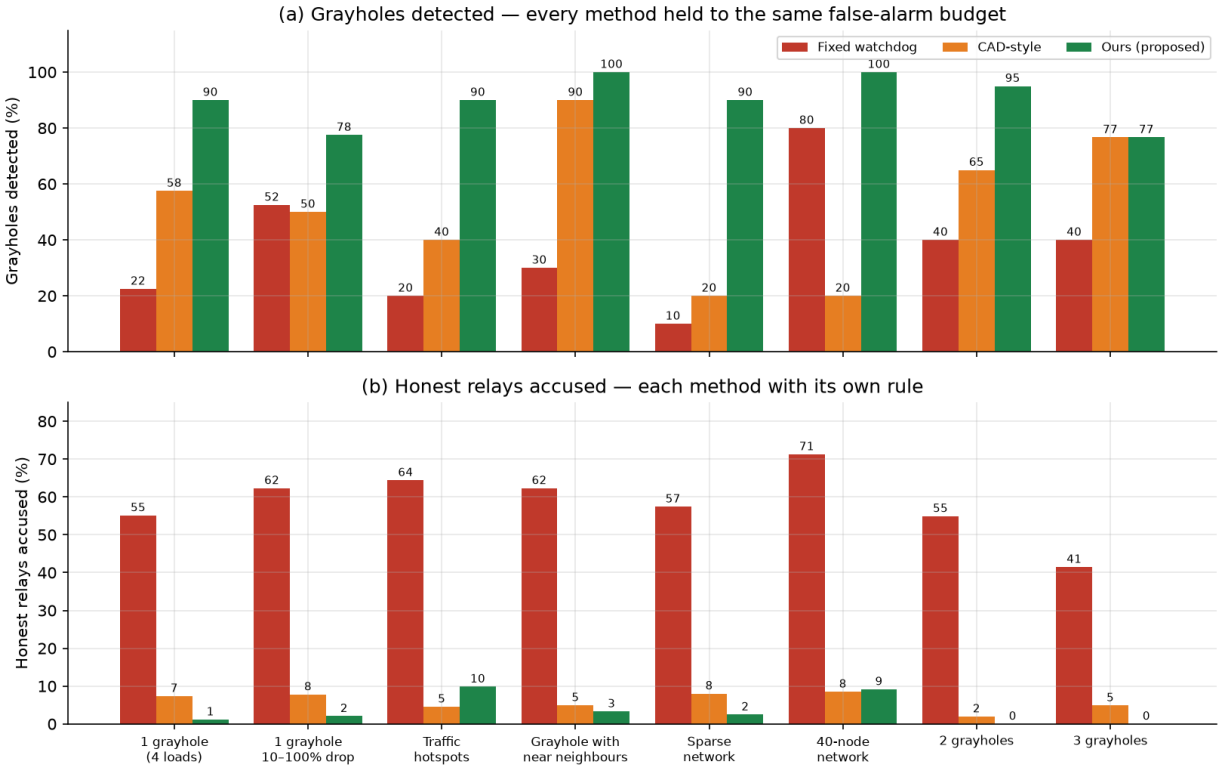


Fig. 10. (a) Grayholes detected with all methods held to the same false-alarm budget; (b) honest relays accused with each method's own rule.

C. Comparison with the Baselines

Figure 8 compares the baseline and the proposed approach in the live network, pooled over every setting. Under attack the fixed watchdog catches every grayhole only by accusing 7.2 honest nodes per run, and routing around them costs most of what it gains: delivery rises only from 45.6% to 47.0%. The CAD-style detector accuses fewer honest nodes (0.8) but misses 42% of the grayholes. Ours catches 95% of the grayholes while accusing 0.3 honest nodes per run, and gives the highest delivery ratio (49.7%) and throughput (200.5 against 186.8 packets per hour without defence).

The reason is shown in Figure 9. With no attacker at all, the fixed watchdog accuses 1.0 honest relay per run at the lowest load and 6.2 at the highest, because collisions and half-duplex deafness grow with traffic and are counted as drops. The CAD-style detector accuses 0.3–0.7, and ours 0–0.1.

Figure 10 is a fair contest: every method's threshold is tuned on the separate tuning networks so that at most 5% of honest relays are accused without an attacker. Ours then detects 90% of single grayholes (CAD-style 58%, fixed 22%), 95% with

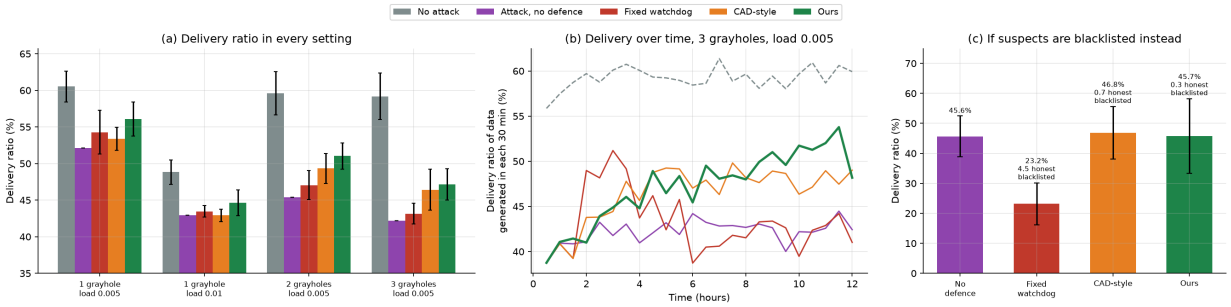


Fig. 11. Live network: (a) delivery ratio per setting, error bars are paired 95% intervals against no defence; (b) delivery over time with three grayholes; (c) delivery when suspects are blacklisted.

two grayholes (65%, 40%) and 77% with three (77%, 40%). With their own rules, the fixed watchdog reaches 100% only by accusing 41–71% of honest relays. In the traffic-hotspot and 40-node networks, ours accuses about 10% of honest relays, which is more than the fixed watchdog at the same budget.

Figure 11 breaks the live-network delivery down by setting. Ours has the highest delivery in all four settings. Compared with no defence on the same networks it gains 4.0, 1.7, 5.6 and 4.9 percentage points, and compared with the fixed watchdog 4.0 points with two and with three grayholes. Against the CAD-style detector ours is ahead by 0.7–2.7 points, which is within the uncertainty. When suspects are blacklisted instead of avoided, every honest node the fixed watchdog accuses is lost together with the nodes behind it, and its delivery falls to 23.2%, against 45.6% without defence.

Finally, each part of the method was removed in turn at the same budget. Without baseline cleaning, detection with three grayholes falls from 23 of 30 to 17 of 30. Without the observability filter, detection is similar, but honest accusations rise slightly (1.5% against 1.1% in the load sweep); with two-round confirmation in place, the filter's contribution in these networks is therefore small.

VI. Conclusions and Future work

We have demonstrated a grayhole attack model as well as an observability-aware, trust-based detection strategy for underwater acoustic networks, implemented in UnetStack and compared with a fixed-threshold watchdog and a CAD-style detector. A performance evaluation of all of them has been done and the results shown in graphs: the detector caught 95% of the grayholes while accusing 0.3 honest nodes per run, against 7.2 for the fixed watchdog, and gave the highest delivery ratio.

The limitations of this paper are as follows. The topologies are static, node positions and the distance–loss curve are assumed known, and every monitor reports truthfully. The baselines are adaptations, not full reproductions of the published systems, and route updates are free in the simulation. In hotspot and 40-node networks ours accuses about 10% of honest relays.

Future implementations can work on making the detector robust to monitors that send false reports, and to mobile nodes and location errors. Work can be done to learn the loss curve online instead of assuming it. Finally, the above approach is only a detection and avoidance scheme. Work can be done to implement an isolation approach that will run on UnetStack without cutting off the nodes behind a grayhole.

Acknowledgment

The authors thank the course staff of CS301 and Mohit sir, Department of Computer Science and Engineering, NITK Surathkal, for their guidance.

References

[1] I. F. Akyildiz, D. Pompili, and T. Melodia, "Underwater acoustic sensor networks: research challenges," *Ad Hoc Networks*, vol. 3, no. 3, pp. 257–279, 2005, doi: 10.1016/j.adhoc.2005.01.004.

[2] C. Karlof and D. Wagner, "Secure routing in wireless sensor networks: attacks and countermeasures," *Ad Hoc Networks*, vol. 1, no. 2–3, pp. 293–315, 2003, doi: 10.1016/S1570-8705(03)00008-8.

[3] M. C. Domingo, "Securing underwater wireless communication networks," *IEEE Wireless Communications*, vol. 18, no. 1, pp. 22–28, 2011, doi: 10.1109/MWC.2011.5714022.

[4] S. Jiang, "On securing underwater acoustic networks: a survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 729–752, 2019, doi: 10.1109/COMST.2018.2864127.

[5] S. Marti, T. J. Giuli, K. Lai, and M. Baker, "Mitigating routing misbehavior in mobile ad hoc networks," in *Proc. 6th ACM MobiCom*, 2000, pp. 255–265, doi: 10.1145/345910.345955.

[6] D. M. Shila, Y. Cheng, and T. Anjali, "Mitigating selective forwarding attacks with a channel-aware approach in WMNs," *IEEE Transactions on Wireless Communications*, vol. 9, no. 5, pp. 1661–1675, 2010, doi: 10.1109/TWC.2010.05.090700.

[7] G. Han, J. Jiang, L. Shu, and M. Guizani, "An attack-resistant trust model based on multidimensional trust metrics in underwater acoustic sensor network," *IEEE Transactions on Mobile Computing*, vol. 14, no. 12, pp. 2447–2459, 2015, doi: 10.1109/TMC.2015.2402120.

[8] C. Liu, J. Ye, F. An, and W. Jiang, "An adaptive trust evaluation model for detecting abnormal nodes in underwater acoustic sensor networks," *Sensors*, vol. 24, no. 9, art. 2880, 2024, doi: 10.3390/s24092880.

[9] D. Han, X. Du, L. Wang, X. Liu, and X. Tian, "Trust-aware and fuzzy logic-based reliable layering routing protocol for underwater acoustic networks," *Sensors*, vol. 23, no. 23, art. 9323, 2023, doi: 10.3390/s23239323.

[10] B. R. Chandavarkar and N. Suresh, "Demonstration of selective forwarding attack and detection mechanism in UnetStack3," in *Proc. 12th Int. Conf. Computing Communication and Networking Technologies (ICCCNT)*, 2021, pp. 1–6, doi: 10.1109/ICCCNT51525.2021.9580125.

[11] M. Chitre, R. Bhatnagar, and W.-S. Soh, "UnetStack: an agent-based software stack and simulator for underwater networks," in *Proc. IEEE OCEANS*, St. John's, 2014, doi: 10.1109/OCEANS.2014.7003044.

[12] UnetStack, "Unet handbook: physical service," https://unetstack.net/handbook/305_phy.html, accessed 12 Oct. 2026.